

Michigan Tech

SVD and Newton

Including chain rule

Allan Struthers

Math Sciences

MA4535 — Nonlinear Dynamics & Chaos | Lecture 4 | September 9, 2026

Michigan Tech

SVD and Newton

Including chain rule

Allan Struthers

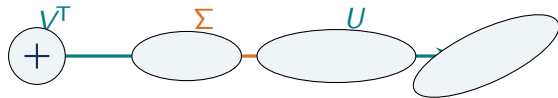
Math Sciences

MA4535 — Nonlinear Dynamics & Chaos | Lecture 4 | September 9, 2026

Every linear map is a rotation, a coordinate-wise scaling, and another rotation.

SVD factorization

$$A = U\Sigma V^T$$

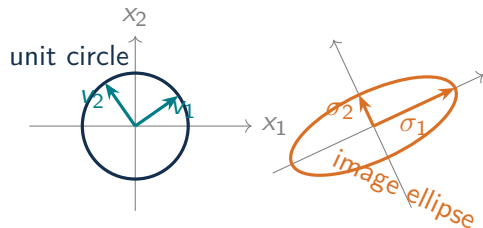


- V^T changes into the right-singular-vector coordinates.
- Σ stretches the coordinate axes by σ_1, σ_2 .
- U rotates into the left-singular-vector coordinates.

SVD Geometry: The unit circle becomes an ellipse

For any unit vector x , the point Ax lies on an ellipse. The longest and shortest radii of that ellipse are the singular values.

$$\sigma_1 = \max_{\|x\|=1} \|Ax\|, \quad \sigma_2 = \min_{\|x\|=1} \|Ax\|$$

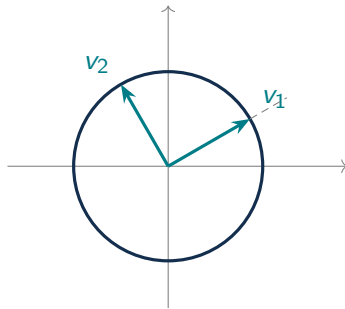


Right singular vectors choose input directions

The columns of $V = [v_1 \ v_2]$ form an orthonormal basis of the input space.

$$Av_i = \sigma_i u_i$$

Each v_i identifies an input direction that A maps exactly onto a principal axis of the output ellipse.



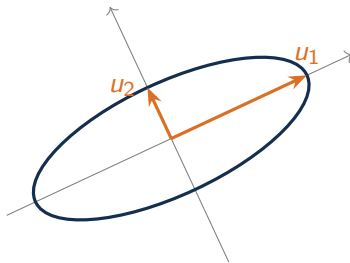
orthogonal input directions

Left singular vectors choose output directions

The columns of $U = [u_1 \ u_2]$ point along the orthonormal principal axes of the output ellipse.

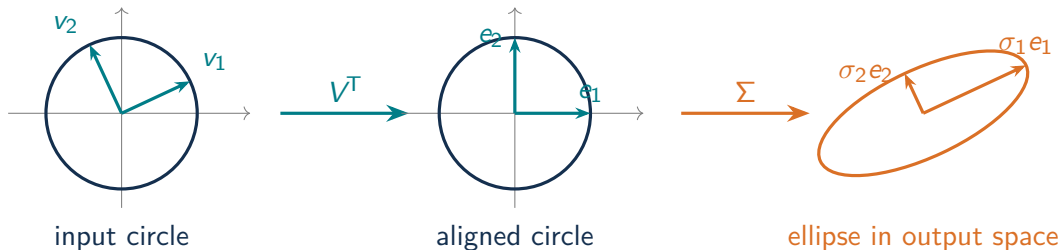
$$Av_1 = \sigma_1 u_1, \quad Av_2 = \sigma_2 u_2$$

The direction can rotate, while the length changes by the corresponding singular value.



orthogonal output directions

A complete geometric picture

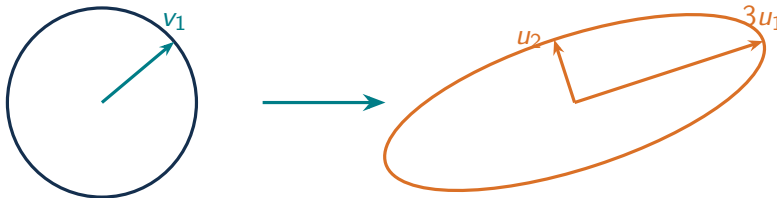


A numerical example

Let

$$A = U \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} V^T.$$

Then A maps the unit circle to an ellipse with semi-axis lengths 3 and 1.



The largest amplification occurs along v_1 ; the weakest occurs along v_2 .

Eigenvalues and singular values

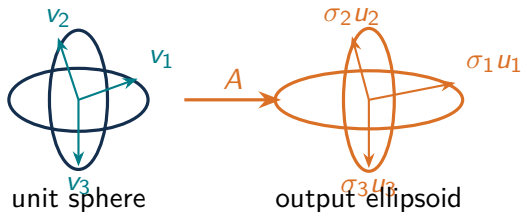
	Eigenvalues	Singular values
Matrix	Defined for square matrices	Defined any $m \times n$ matrix
Equation	$Ax = \lambda x$	$Av_i = \sigma_i u_i$
Values	Real or complex	Real and nonnegative
Directions	Matrices can have complex and missing eigenvectors!	Singular vector basis always exist!
Meaning	Eigenvectors v are parallel to Av	Singular vectors are orthogonal directions with simple stretches.

SVD geometry in 3D

The unit sphere maps to an ellipsoid.

$$A = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^T$$

Right singular vectors v_1, v_2, v_3 are orthogonal input directions. Left singular vectors u_1, u_2, u_3 are the orthogonal axes of the output ellipsoid.



Why the SVD matters

- **Geometry:** It identifies the directions that a linear map stretches most and least.
- **Stability:** A very small σ_i marks a direction that the map nearly collapses.
- **Approximation:** Large singular values give the best low-rank approximation.

One sentence

The SVD expresses a matrix as orthogonal input directions, independent stretches, and orthogonal output directions.

Chain-rule review: 1D

Let

$$y = f(u), \quad u = g(x),$$

so that $y = (f \circ g)(x) = f(g(x))$. A change in x first changes u , and the change in u then changes y .

The chain rule multiplies the two local rates of change:

$$\boxed{\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}}$$

or, in function notation,

$$\boxed{(f \circ g)'(x) = f'(g(x))g'(x)}.$$

The derivative of the outer function is evaluated at the inner value $g(x)$.

Chain-rule review: 1D example

Consider

$$h(x) = \sin(x^2 + 1).$$

Separate the composition into

$$u = g(x) = x^2 + 1, \quad h = f(u) = \sin u.$$

Then

$$\frac{dh}{du} = \cos u, \quad \frac{du}{dx} = 2x.$$

Therefore

$$h'(x) = \cos(x^2 + 1)(2x).$$

At $x = x_0$, the corresponding linear approximation is

$$h(x_0 + \Delta x) \approx h(x_0) + h'(x_0)\Delta x.$$

Chain-rule review: A scalar function two intermediate variables

Let

$$z = f(x, y), \quad x = x(t), \quad y = y(t).$$

Both x and y transmit the effect of changing t . The total derivative is

$$\boxed{\frac{dz}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}}.$$

Using the gradient and the velocity vector,

$$\nabla f(x, y) = \begin{bmatrix} f_x(x, y) \\ f_y(x, y) \end{bmatrix}, \quad \mathbf{r}'(t) = \begin{bmatrix} x'(t) \\ y'(t) \end{bmatrix},$$

the same formula becomes

$$\boxed{\frac{d}{dt} f(x(t), y(t)) = \nabla f(x(t), y(t))^T \mathbf{r}'(t)}.$$

Chain-rule review: 2D

Let $\mathbf{G} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ and $\mathbf{F} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$:

$$\mathbf{u} = \mathbf{G}(x, y) = \begin{bmatrix} g_1(x, y) \\ g_2(x, y) \end{bmatrix}, \quad \mathbf{H}(x, y) = \mathbf{F}(\mathbf{G}(x, y)).$$

The Jacobian form of the chain rule is

$$J_{\mathbf{H}}(x, y) = J_{\mathbf{F}}(\mathbf{G}(x, y)) J_{\mathbf{G}}(x, y).$$

Fully expanded, this is

$$\begin{bmatrix} \frac{\partial H_1}{\partial x} & \frac{\partial H_1}{\partial y} \\ \frac{\partial H_2}{\partial x} & \frac{\partial H_2}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial F_1}{\partial u_1} & \frac{\partial F_1}{\partial u_2} \\ \frac{\partial F_2}{\partial u_1} & \frac{\partial F_2}{\partial u_2} \end{bmatrix}_{\mathbf{u}=\mathbf{G}(x,y)} \begin{bmatrix} \frac{\partial g_1}{\partial x} & \frac{\partial g_1}{\partial y} \\ \frac{\partial g_2}{\partial x} & \frac{\partial g_2}{\partial y} \end{bmatrix}.$$

Chain-rule review: A scalar function along a 3D curve

Let

$$w = f(x, y, z), \quad x = x(t), \quad y = y(t), \quad z = z(t).$$

The three-dimensional chain rule is

$$\boxed{\frac{dw}{dt} = f_x \frac{dx}{dt} + f_y \frac{dy}{dt} + f_z \frac{dz}{dt}},$$

where the partial derivatives are evaluated at $(x(t), y(t), z(t))$. With

$$\nabla f = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}, \quad \mathbf{r}'(t) = \begin{bmatrix} x'(t) \\ y'(t) \\ z'(t) \end{bmatrix},$$

we obtain the compact formula

$$\boxed{\frac{d}{dt} f(\mathbf{r}(t)) = \nabla f(\mathbf{r}(t))^T \mathbf{r}'(t)}.$$

Chain-rule review: The Jacobian in 3D

For $\mathbf{g}: \mathbb{R}^3 \rightarrow \mathbb{R}^3$, $\mathbf{f}: \mathbb{R}^3 \rightarrow \mathbb{R}^3$, and $\mathbf{h} = \mathbf{f} \circ \mathbf{g}$,

$$\boxed{J_{\mathbf{h}}(\mathbf{x}) = J_{\mathbf{f}}(\mathbf{g}(\mathbf{x}))J_{\mathbf{g}}(\mathbf{x})}.$$

Each Jacobian has the explicit form

$$J_{\mathbf{f}}(\mathbf{u}) = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \frac{\partial f_1}{\partial u_2} & \frac{\partial f_1}{\partial u_3} \\ \frac{\partial f_2}{\partial u_1} & \frac{\partial f_2}{\partial u_2} & \frac{\partial f_2}{\partial u_3} \\ \frac{\partial f_3}{\partial u_1} & \frac{\partial f_3}{\partial u_2} & \frac{\partial f_3}{\partial u_3} \end{bmatrix}, \quad J_{\mathbf{g}}(\mathbf{x}) = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \frac{\partial g_1}{\partial x_3} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \frac{\partial g_2}{\partial x_3} \\ \frac{\partial g_3}{\partial x_1} & \frac{\partial g_3}{\partial x_2} & \frac{\partial g_3}{\partial x_3} \end{bmatrix}.$$

Order matters: Outer map Jacobian multiplies the inner map Jacobian.

Chain-rule review: Linearization

For smooth $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ the Jacobian gives the 1st order response to a small change $\mathbf{s}$

$$\mathbf{F}(\mathbf{x} + \mathbf{s}) = \mathbf{F}(\mathbf{x}) + J(\mathbf{x})\mathbf{s} + o(\|\mathbf{s}\|).$$

If $\mathbf{x} = \mathbf{x}(t)$, the chain rule gives

$$\frac{d}{dt}\mathbf{F}(\mathbf{x}(t)) = J(\mathbf{x}(t))\mathbf{x}'(t).$$

The Jacobian maps small input changes to corresponding first-order output changes. Newton's method chooses $\mathbf{s}_k$ so that the linear model of $\mathbf{F}(\mathbf{x}_k + \mathbf{s}_k)$ equals zero.

Newton's method for scalar $f(x) = 0$

Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be differentiable. We seek a root x^* satisfying

$$f(x^*) = 0.$$

At the current iterate x_k , replace f by its tangent-line approximation:

$$f(x_k + s) \approx f(x_k) + f'(x_k)s.$$

Choose the step s_k so that the linear approximation vanishes:

$$f(x_k) + f'(x_k)s_k = 0.$$

Provided $f'(x_k) \neq 0$,

$$s_k = -\frac{f(x_k)}{f'(x_k)}.$$

1D Newton iteration

Since $x_{k+1} = x_k + s_k$, Newton's iteration is

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}, \quad k = 0, 1, 2, \dots$$

Algorithm

Choose an initial value x_0 . For each k , evaluate $f(x_k)$ and $f'(x_k)$, solve for the Newton step, and update $x_{k+1} = x_k + s_k$.

Common stopping tests include

$$|f(x_k)| \leq \varepsilon_f, \quad |x_{k+1} - x_k| \leq \varepsilon_x(1 + |x_{k+1}|).$$

When $f'(x_k) \approx 0$ this can produce excessively large steps and fail!

2D Newton

Consider

$$f_1(x, y) = 0, \quad f_2(x, y) = 0.$$

Write the system as a vector equation

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(x, y) \\ f_2(x, y) \end{bmatrix} = \mathbf{0}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}.$$

At iteration k , let

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}, \quad \mathbf{s}_k = \begin{bmatrix} \Delta x_k \\ \Delta y_k \end{bmatrix}.$$

The first-order approximation is

$$\mathbf{F}(\mathbf{x}_k + \mathbf{s}_k) \approx \mathbf{F}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\mathbf{s}_k.$$

2D Jacobian matrix

The Jacobian contains every first partial derivative:

$$J(x, y) = \begin{bmatrix} \frac{\partial f_1}{\partial x}(x, y) & \frac{\partial f_1}{\partial y}(x, y) \\ \frac{\partial f_2}{\partial x}(x, y) & \frac{\partial f_2}{\partial y}(x, y) \end{bmatrix}$$

The Newton step solves the linear system

$$J(x_k, y_k) \begin{bmatrix} \Delta x_k \\ \Delta y_k \end{bmatrix} = - \begin{bmatrix} f_1(x_k, y_k) \\ f_2(x_k, y_k) \end{bmatrix}.$$

Then update both variables:

$$x_{k+1} = x_k + \Delta x_k, \quad y_{k+1} = y_k + \Delta y_k.$$

2D Newton example

Let

$$f_1(x, y) = x^2 + y^2 - 4, \quad f_2(x, y) = x - y - 1.$$

Then

$$\mathbf{F}(x, y) = \begin{bmatrix} x^2 + y^2 - 4 \\ x - y - 1 \end{bmatrix}, \quad J(x, y) = \begin{bmatrix} 2x & 2y \\ 1 & -1 \end{bmatrix}.$$

At (x_k, y_k) , solve

$$\begin{bmatrix} 2x_k & 2y_k \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \Delta x_k \\ \Delta y_k \end{bmatrix} = - \begin{bmatrix} x_k^2 + y_k^2 - 4 \\ x_k - y_k - 1 \end{bmatrix},$$

and set

$$(x_{k+1}, y_{k+1}) = (x_k + \Delta x_k, y_k + \Delta y_k).$$

3D Jacobian

For

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(x, y, z) \\ f_2(x, y, z) \\ f_3(x, y, z) \end{bmatrix}$$

the Jacobian is the 3×3 matrix

$$J(x, y, z) = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial z} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial z} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} & \frac{\partial f_3}{\partial z} \end{bmatrix}.$$

The rows correspond to equations. The columns correspond to the variables.

3D Newton step

At iteration k , write

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix}, \quad \mathbf{s}_k = \begin{bmatrix} \Delta x_k \\ \Delta y_k \\ \Delta z_k \end{bmatrix}.$$

Solve the explicit linear system

$$J(\mathbf{x}_k) \begin{bmatrix} \Delta x_k \\ \Delta y_k \\ \Delta z_k \end{bmatrix} = - \begin{bmatrix} f_1(\mathbf{x}_k) \\ f_2(\mathbf{x}_k) \\ f_3(\mathbf{x}_k) \end{bmatrix}.$$

Then

$$\boxed{\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k}$$

or equivalently $x_{k+1} = x_k + \Delta x_k$, $y_{k+1} = y_k + \Delta y_k$, $z_{k+1} = z_k + \Delta z_k$.

The general problem in n dimensions

$$\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

The Jacobian $J(\mathbf{x}) \in \mathbb{R}^{n \times n}$ is

$$\boxed{J_{ij}(\mathbf{x}) = \frac{\partial f_i}{\partial x_j}(\mathbf{x}), \quad 1 \leq i, j \leq n} \quad \text{so} \quad J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}.$$

Newton in $\mathbb{R}^n$

Given

$$\mathbf{x}_k = \begin{bmatrix} x_{1,k} \\ x_{2,k} \\ \vdots \\ x_{n,k} \end{bmatrix},$$

compute the Newton step $\mathbf{s}_k$ from

$$\boxed{J(\mathbf{x}_k)\mathbf{s}_k = -\mathbf{F}(\mathbf{x}_k)} \quad \text{and update} \quad \boxed{\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k}.$$

Use linear-solve rather than explicitly form the inverse $J(\mathbf{x}_k)^{-1}$. It is more stable, accurate, and efficient. Brevity is the only real advantage of

$$\boxed{\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1}\mathbf{f}(\mathbf{x}_k)}.$$

Rapid terminal convergence

Suppose $f(\mathbf{x}^*) = 0$ and $J(\mathbf{x}^*)$ is smooth and invertible near $\mathbf{x}^*$ then good initial guesses $\mathbf{x}_0$ converge quadratically. For some $C > 0$ and $\epsilon > 0$

$$\boxed{\|\mathbf{x}_0 - \mathbf{x}^*\| < \epsilon} \implies \boxed{\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq C \|\mathbf{x}_k - \mathbf{x}^*\|^2}.$$

This terminal convergence is fast, roughly doubling the # of correct digits at each step! If $\det(J(\mathbf{x}^*)) = 0$ Newton's method can fail or be slow.

Initial guesses $\mathbf{x}_0$ and safeguards

Newton's method has great local behavior. Iterates from poor $\mathbf{x}_0$ can diverge, cycle, approach another root, or hit singular Jacobians.

Practical guidance

Choose $\mathbf{x}_0$ from a graph or simple approximation. Monitor both the residual $\|\mathbf{F}(\mathbf{x}_k)\|$ and step $\|\mathbf{s}_k\|$.

Line searches and trust-region methods choose controlled steps until rapid terminal convergence takes over.

The pattern at every dimension

$$\text{Linearize: } \mathbf{F}(\mathbf{x}_k + \mathbf{s}_k) \approx \mathbf{F}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\mathbf{s}_k,$$

$$\text{Solve: } \mathbf{J}(\mathbf{x}_k)\mathbf{s}_k = -\mathbf{F}(\mathbf{x}_k),$$

$$\text{Update: } \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k.$$

The scalar derivative $f'(x_k)$ becomes the Jacobian matrix $\mathbf{J}(\mathbf{x}_k)$. The scalar division becomes a linear solve.

Newton Summary

- Newton $x_{k+1} = x_k + J_k^{-1}f(x_k)$ solves $f(x_k) = 0$ given a good initial condition x_0 .
- Every technical software tool has Newton like solvers: FindRoot in Mathematica, NonlinearSolve in Julia, fsolve in matlab, etc.